UADE
UNA GRAN UNIVERSIDAD
FUNDACIÓN UADE

TRABAJO PRÁCTICO

Base de datos II

Integrantes del Grupo:

Perez, Thomas

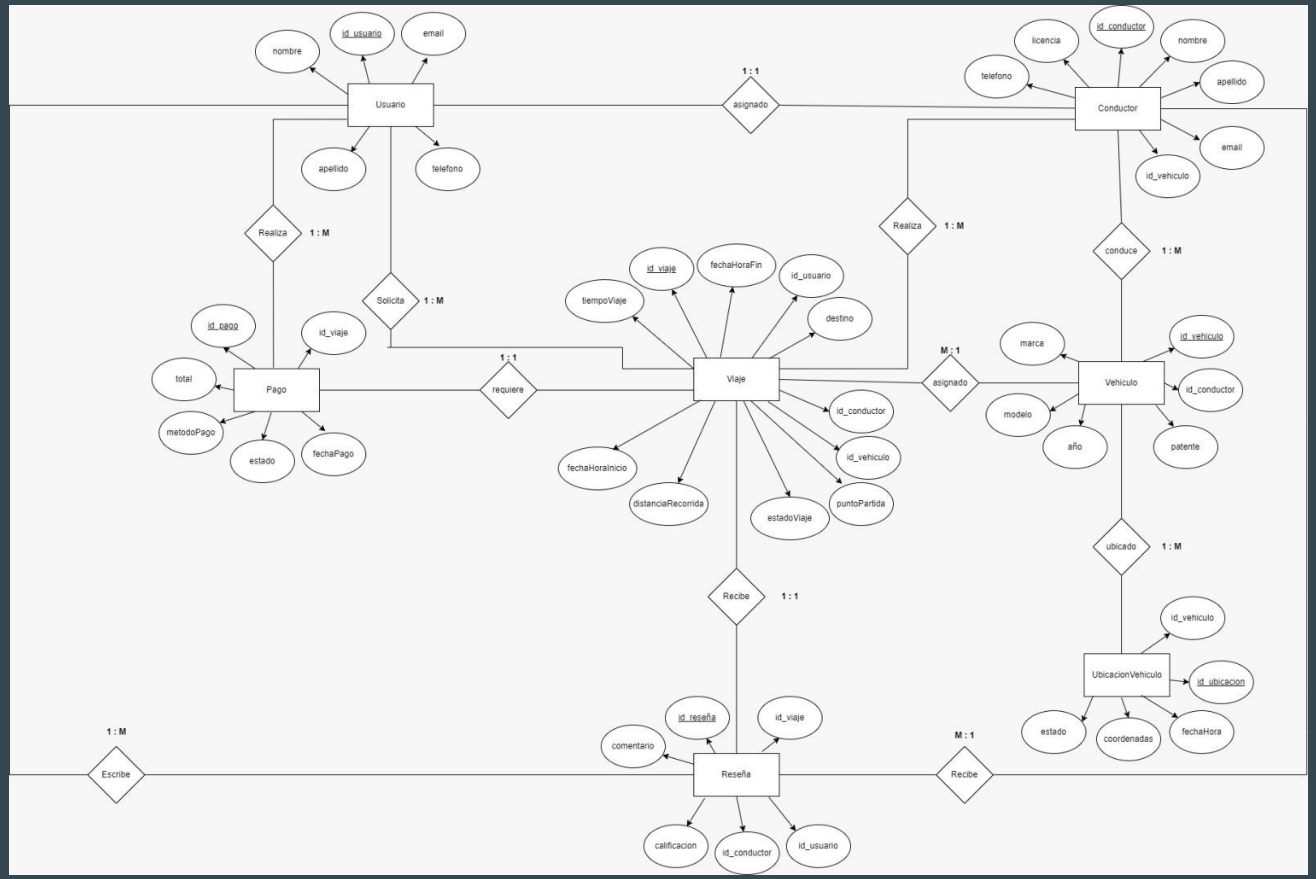
Sistema de Gestión de Datos
Uber

En el sector de tecnología surge la necesidad de diseñar un modelo de datos para una plataforma de transporte.

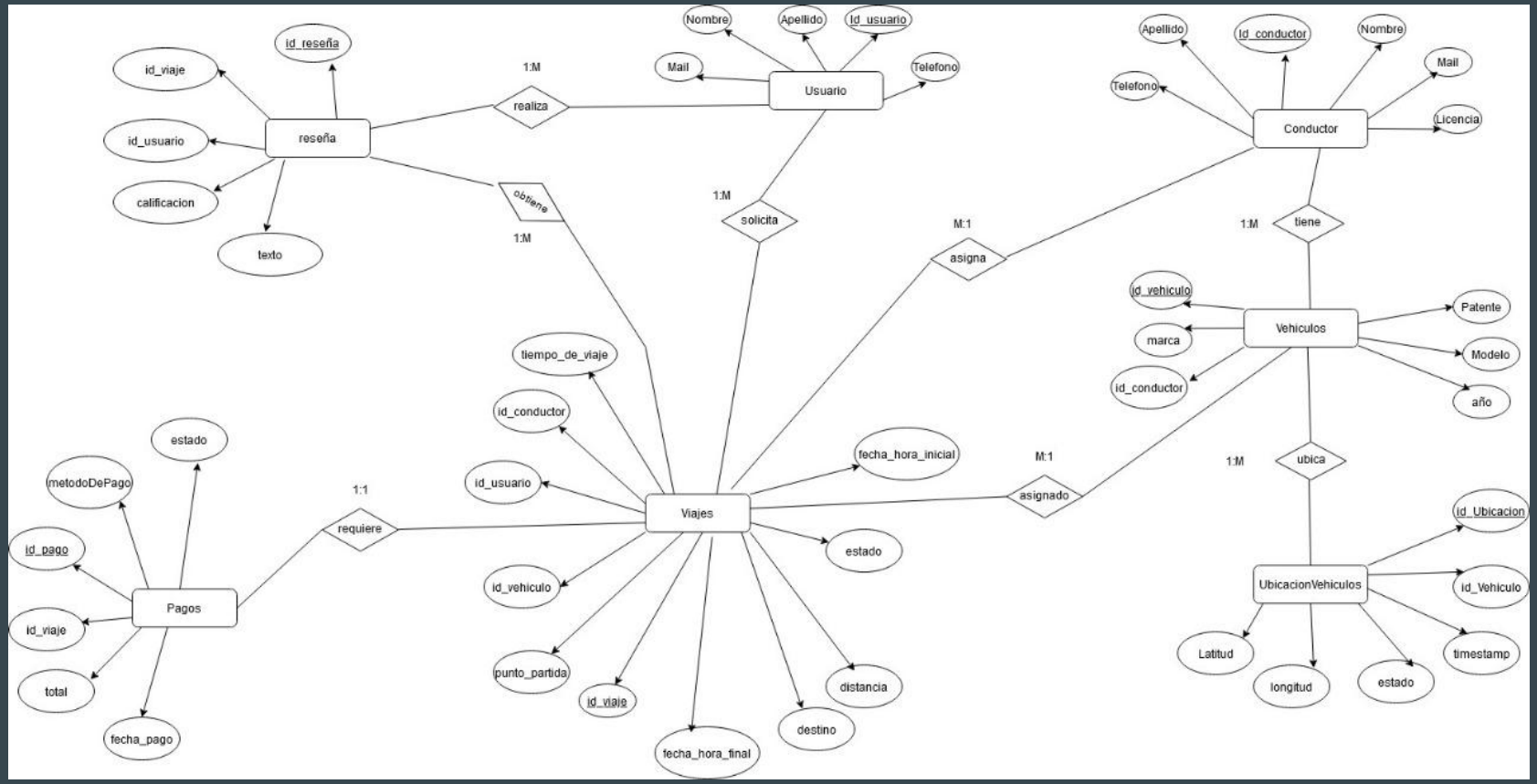
Objetivos:

- Gestionar información sobre usuarios, conductores, vehículos y viajes
- Consultar detalles de los viajes
- Visualizar las interacciones entre los usuarios y conductores
- Gestionar los pagos

DER recibido



DER actualizado



Modificaciones al DER recibido

- Eliminamos relaciones que creíamos innecesarias
- La relación entre usuario con conductor,usuarios con pagos y conductores con reseña
- Cambiamos los atributos de la tabla ubicacionVehiculo
- cambiamos coordenadas por latitud y longitud

UBER en SQL

```sql
create table Usuarios(
id_usuario int,
nombre varchar(100),
apellido varchar(100),
mail varchar(100),
telefono int,
constraint Usuarios_PK primary key(id_usuario)
);
```

```sql
INSERT INTO Usuarios (id_usuario, nombre, apellido, mail, telefono) VALUES (1, 'Juan', 'Pérez', 'juan.perez@example.com', 123456789);
INSERT INTO Usuarios (id_usuario, nombre, apellido, mail, telefono) VALUES (2, 'María', 'Gómez', 'maria.gomez@example.com', 987654321);
INSERT INTO Usuarios (id_usuario, nombre, apellido, mail, telefono) VALUES (3, 'Carlos', 'López', 'carlos.lopez@example.com', 456123789);
INSERT INTO Usuarios (id_usuario, nombre, apellido, mail, telefono) VALUES (4, 'Ana', 'Torres', 'ana.torres@example.com', 321654987);
INSERT INTO Usuarios (id_usuario, nombre, apellido, mail, telefono) VALUES (5, 'Laura', 'Fernández', 'laura.fernandez@example.com', 654789123);
```

UBER en SQL

```sql
create table Conductor(
id_conductor int,
nombre varchar(100),
apellido varchar(100),
mail varchar(100),
telefono int,
constraint Conductors_PK primary key(id_conductor)
);
```

```sql
INSERT INTO Usuarios (id_usuario, nombre, apellido, mail, telefono) VALUES (1, 'Juan', 'Pérez', 'juan.perez@example.com', 123456789);
INSERT INTO Usuarios (id_usuario, nombre, apellido, mail, telefono) VALUES (2, 'María', 'Gómez', 'maria.gomez@example.com', 987654321);
INSERT INTO Usuarios (id_usuario, nombre, apellido, mail, telefono) VALUES (3, 'Carlos', 'López', 'carlos.lopez@example.com', 456123789);
INSERT INTO Usuarios (id_usuario, nombre, apellido, mail, telefono) VALUES (4, 'Ana', 'Torres', 'ana.torres@example.com', 321654987);
INSERT INTO Usuarios (id_usuario, nombre, apellido, mail, telefono) VALUES (5, 'Laura', 'Fernández', 'laura.fernandez@example.com', 654789123);
```

UBER en SQL

```sql
create table Veiculo(
id_veiculo int,
id_conductor int,
marca varchar(100),
patente varchar(100),
modelo varchar(100),
constraint Veiculo_PK primary key(id_veiculo),
constraint Veiculo_Conductor_FK foreign key (id_conductor) references Conductor(id_conductor)
);
```

```sql
INSERT INTO Veiculo (id_veiculo, id_conductor, marca, patente, modelo) VALUES (1, 1, 'Toyota', 'ABC123', 'Corolla');
INSERT INTO Veiculo (id_veiculo, id_conductor, marca, patente, modelo) VALUES (2, 2, 'Ford', 'DEF456', 'Fiesta');
INSERT INTO Veiculo (id_veiculo, id_conductor, marca, patente, modelo) VALUES (3, 3, 'Chevrolet', 'GHI789', 'Onix');
INSERT INTO Veiculo (id_veiculo, id_conductor, marca, patente, modelo) VALUES (4, 4, 'Nissan', 'JKL012', 'Versa');
INSERT INTO Veiculo (id_veiculo, id_conductor, marca, patente, modelo) VALUES (5, 5, 'Hyundai', 'MNO345', 'Elantra');
INSERT INTO Veiculo (id_veiculo, id_conductor, marca, patente, modelo) VALUES (6, 1, 'Toyota', 'ABC12D', 'Corolla');
```

UBER en SQL

```sql
create table Viaje(
id_viaje int,
id_usuario int,
id_conductor int,
id_veiculo int,
punto_partida VARCHAR(100),
destino VARCHAR(100),
distancia DECIMAL(5, 2),
tiempo_viaje TIME,
estado varchar(100),
fecha_hora_inicio DATETIME,
fecha_hora_final DATETIME,
constraint Viaje_PK primary key(id_viaje),
constraint Usuarios_Viaje_FK foreign key(id_usuario) references Usuarios(id_usuario),
constraint Conductor_Viaje_FK foreign key(id_conductor) references Conductor(id_conductor),
constraint Veiculo_Viaje_FK foreign key(id_veiculo) references Veiculo(id_veiculo)
);
```

```sql
INSERT INTO Viaje (id_viaje, id_usuario, id_conductor, id_veiculo, punto_partida, destino, distancia, tiempo_viaje, estado, fecha_hora_inicio, fecha_hora_final) VALUES (1, 1, 1, 1, 'Centro', 'Aeropuerto', 25.00, '00:30:00', 'Completado', '2024-10-01 10:00:00', '2024-10-01 10:30:00');
INSERT INTO Viaje (id_viaje, id_usuario, id_conductor, id_veiculo, punto_partida, destino, distancia, tiempo_viaje, estado, fecha_hora_inicio, fecha_hora_final) VALUES (2, 2, 2, 2, 'Plaza', 'Estadio', 10.50, '00:15:00', 'Completado', '2024-10-01 11:00:00', '2024-10-01 11:15:00');
INSERT INTO Viaje (id_viaje, id_usuario, id_conductor, id_veiculo, punto_partida, destino, distancia, tiempo_viaje, estado, fecha_hora_inicio, fecha_hora_final) VALUES (3, 3, 3, 3, 'Parque', 'Museo', 5.00, '00:10:00', 'Completado', '2024-10-01 12:00:00', '2024-10-01 12:10:00');
INSERT INTO Viaje (id_viaje, id_usuario, id_conductor, id_veiculo, punto_partida, destino, distancia, tiempo_viaje, estado, fecha_hora_inicio, fecha_hora_final) VALUES (4, 4, 4, 4, 'Cine', 'Casa', 2.75, '00:05:00', 'Completado', '2024-10-01 13:00:00', '2024-10-01 13:05:00');
INSERT INTO Viaje (id_viaje, id_usuario, id_conductor, id_veiculo, punto_partida, destino, distancia, tiempo_viaje, estado, fecha_hora_inicio, fecha_hora_final) VALUES (5, 5, 5, 5, 'Oficina', 'Centro', 3.50, '00:08:00', 'Completado', '2024-10-01 14:00:00', '2024-10-01 14:08:00');
INSERT INTO Viaje (id_viaje, id_usuario, id_conductor, id_veiculo, punto_partida, destino, distancia, tiempo_viaje, estado, fecha_hora_inicio, fecha_hora_final) VALUES (6, 1, 1, 1, 'Centro', 'Aeropuerto', 25.00, '00:30:00', 'Completado', '2024-10-01 10:00:00', '2024-10-01 10:30:00');
INSERT INTO Viaje (id_viaje, id_usuario, id_conductor, id_veiculo, punto_partida, destino, distancia, tiempo_viaje, estado, fecha_hora_inicio, fecha_hora_final) VALUES (7, 1, 1, 1, 'Centro', 'Aeropuerto', 25.00, '00:30:00', 'Completado', '2024-7-01 10:00:00', '2024-7-01 10:30:00');
INSERT INTO Viaje (id_viaje, id_usuario, id_conductor, id_veiculo, punto_partida, destino, distancia, tiempo_viaje, estado, fecha_hora_inicio, fecha_hora_final) VALUES (8, 1, 6, 1, 'Centro', 'Aeropuerto', 25.00, '00:30:00', 'Completado', '2024-7-01 10:00:00', '2024-7-01 10:30:00');
```

UBER en SQL

```sql
create table Reseña(
id_reseña int,
id_usuario int,
id_viaje int,
calificacion int,
reseña varchar(200),
constraint Reseña_PK primary key(id_reseña),
constraint Reseña_Usuarios_FK foreign key(id_Usuario) references Usuarios(id_usuario),
constraint Reseña_Viaje_FK foreign key(id_viaje) references Viaje(id_viaje)
);
```

```sql
INSERT INTO Reseña (id_reseña, id_usuario, id_viaje, calificacion, reseña) VALUES (1, 1, 1, 5, 'Excelente servicio!');
INSERT INTO Reseña (id_reseña, id_usuario, id_viaje, calificacion, reseña) VALUES (2, 2, 2, 4, 'Buen viaje, conductor amable.');
INSERT INTO Reseña (id_reseña, id_usuario, id_viaje, calificacion, reseña) VALUES (3, 3, 3, 3, 'Viaje regular, podría mejorar.');
INSERT INTO Reseña (id_reseña, id_usuario, id_viaje, calificacion, reseña) VALUES (4, 4, 4, 2, 'No fue lo que esperaba.');
```

UBER en SQL

```sql
create table Pago(
id_pago int,
id_viaje int,
fechaPago date,
total int,
estado varchar(100),
metodoDePago varchar(100),
constraint Pago_PK primary key(id_pago),
constraint Pago_Viaje_FK foreign key(id_viaje) references Viaje(id_viaje)
);
```

```sql
INSERT INTO Pago (id_pago, id_viaje, fechaPago, total, estado, metodoDePago) VALUES (1, 1, '2024-10-01', 1500, 'Completado', 'Tarjeta');
INSERT INTO Pago (id_pago, id_viaje, fechaPago, total, estado, metodoDePago) VALUES (2, 2, '2024-10-01', 700, 'Completado', 'Efectivo');
INSERT INTO Pago (id_pago, id_viaje, fechaPago, total, estado, metodoDePago) VALUES (3, 3, '2024-10-01', 300, 'Completado', 'Tarjeta');
INSERT INTO Pago (id_pago, id_viaje, fechaPago, total, estado, metodoDePago) VALUES (4, 4, '2024-10-01', 100, 'Completado', 'Efectivo');
INSERT INTO Pago (id_pago, id_viaje, fechaPago, total, estado, metodoDePago) VALUES (5, 5, '2024-10-01', 200, 'Completado', 'Tarjeta');
```

UBER en SQL

```sql
create table ubicacionVeiculo(
id_ubicacion int,
id_veiculo int,
estado varchar(100),
latitud DECIMAL(9, 6),
longitud DECIMAL(9, 6),
timestamp DATETIME,
constraint ubicacionVeiculo_PK primary key(id_ubicacion),
constraint ubicacionVeiculo_Veiculo_FK foreign key(id_veiculo) references Veiculo(id_veiculo)
);
```

```sql
INSERT INTO Pago (id_pago, id_viaje, fechaPago, total, estado, metodoDePago) VALUES (1, 1, '2024-10-01', 1500, 'Completado', 'Tarjeta');
INSERT INTO Pago (id_pago, id_viaje, fechaPago, total, estado, metodoDePago) VALUES (2, 2, '2024-10-01', 700, 'Completado', 'Efectivo');
INSERT INTO Pago (id_pago, id_viaje, fechaPago, total, estado, metodoDePago) VALUES (3, 3, '2024-10-01', 300, 'Completado', 'Tarjeta');
INSERT INTO Pago (id_pago, id_viaje, fechaPago, total, estado, metodoDePago) VALUES (4, 4, '2024-10-01', 100, 'Completado', 'Efectivo');
INSERT INTO Pago (id_pago, id_viaje, fechaPago, total, estado, metodoDePago) VALUES (5, 5, '2024-10-01', 200, 'Completado', 'Tarjeta');
```

UBER en SQL - Casos de uso (1)

```python
#Endpoints para SQL

#¿Cual es el top 3 de usuarios que han dado mas reseñas?
@app.route('/users/top3', methods=['GET'])
def obtener_top_usuarios():
    conn = get_db_connection()
    cursor = conn.cursor()

    cursor.execute("""SELECT TOP 3 id_usuario, COUNT(*) AS total_reseñas
                      FROM Reseña
                      GROUP BY id_usuario
                      ORDER BY total_reseñas DESC;
                      """)
    rows = cursor.fetchall()


    items = []
    for row in rows:
        items.append({"id_usuario": row[0], "total_resenia": row[1]})

    conn.close()

    return jsonify(items)
```

UBER en SQL - Casos de uso (1)

Pretty Raw Preview Visualize JSON


```
[
    {
        "id_usuario": 2,
        "total_resenia": 3
    },
    {
        "id_usuario": 1,
        "total_resenia": 3
    },
    {
        "id_usuario": 4,
        "total_resenia": 2
    }
]
```

UBER en SQL - Casos de uso (2)

```python
#¿Cual es el metodo de pago menos utilizado en la plataforma?
@app.route('/pay-method/lessused', methods=['GET'])
def obtener_metodo_pago_menos_utilizado():
    conn = get_db_connection()
    cursor = conn.cursor()

    cursor.execute("""SELECT TOP 1 metodoDePago, COUNT(*) AS total_uso
                      FROM Pago
                      GROUP BY metodoDePago
                      ORDER BY total_uso ASC;
                      """)
    rows = cursor.fetchall()

    items = []
    for row in rows:
        items.append({"metodoDePago": row[0], "total_uso": row[1]})

    conn.close()

    return jsonify(items)
```

UBER en SQL - Casos de uso (2)

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize JSON


```
[
    {
        "metodoDePago": "Efectivo",
        "total_uso": 2
    }
]
```

UBER en SQL - Casos de uso (3)

```python
#¿Cuales son los conductores que han estado inactivos en el ultimo mes?
@app.route('/drivers/inactive', methods=['GET'])
def obtener_conductores_inactivos():
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute("""SELECT c.id_conductor, c.nombre
                      FROM Conductor c
                      WHERE c.id_conductor NOT IN (
                          SELECT id_conductor
                          FROM Viaje
                          WHERE fecha_hora_final >= DATEADD(DAY, -30, GETDATE()));"""
                      )
    rows = cursor.fetchall()

    items = []
    for row in rows:
        items.append({"id_conductor": row[0], "nombre": row[1]})

    conn.close()

    return jsonify(items)
```

UBER en SQL - Casos de uso (3)

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize JSON


```
[
    {
        "id_conductor": 1,
        "nombre": "Pedro"
    },
    {
        "id_conductor": 2,
        "nombre": "Lucia"
    },
    {
        "id_conductor": 3,
        "nombre": "Javier"
    },
    {
        "id_conductor": 4,
        "nombre": "Elena"
    },
    {
        "id_conductor": 5,
        "nombre": "Mario"
    },
    {
        "id_conductor": 6,
        "nombre": "Maximo"
    },
    {
        "id_conductor": 7,
        "nombre": "Maximo"
    }
]
```

UBER en SQL - Casos de uso (4)

```python
#¿Que conductores y pasajeros han coincidido en mas de 1 viaje en la aplicacion?
@app.route('/coincidencias', methods=['GET'])
def obtener_viajes_comunes():
    conn = get_db_connection()
    cursor = conn.cursor()

    cursor.execute("""SELECT
                        v.id_conductor, v.id_usuario, COUNT(v.id_viaje) AS total_viajes
                        FROM  Viaje v
                        GROUP BY v.id_conductor, v.id_usuario
                        HAVING COUNT(v.id_viaje) > 1;
                        """)
    rows = cursor.fetchall()

    items = []
    for row in rows:
        items.append({"id_coductor": row[0], "id_usuario": row[1], "total_viajes": row[2]})

    conn.close()

    return jsonify(items)
```

UBER en SQL - Casos de uso (4)

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize JSON


```
[
    {
        "id_coductor": 1,
        "id_usuario": 1,
        "total_viajes": 3
    }
]
```

UBER en SQL - Casos de uso (5)

```python
#¿Cuantos autos con terminacion de patente en “D” y de Marca Toyota hay en la plataforma?
@app.route('/cars/toyota', methods=['GET'])
def obtener_autos_toyota():
    conn = get_db_connection()
    cursor = conn.cursor()

    cursor.execute("""SELECT COUNT(*) AS total_veiculos
                      FROM Veiculo
                      WHERE marca = 'Toyota' AND patente LIKE '%D';
                      """)
    rows = cursor.fetchall()

    items = []
    for row in rows:
        items.append({"total Toyotas con D": row[0]})

    conn.close()

    return jsonify(items)
```

UBER en SQL - Casos de uso (5)

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize JSON


```
[
    {
        "total Toyotas con D": 1
    }
]
```

UBER en SQL - Casos de uso (6)

```python
#¿Cual es el tiempo promedio de los viajes?
@app.route('/travels/avgtime', methods=['GET'])
def obtener_tiempo_promedio():
    conn = get_db_connection()
    cursor = conn.cursor()

    cursor.execute("""SELECT
                    CONVERT(VARCHAR, DATEADD(SECOND, AVG(DATEDIFF(SECOND, '00:00:00', tiempo_viaje)), '00:00:00'), 108) AS TiempoPromedio
                    FROM Viaje;
                    """)
    rows = cursor.fetchall()

    items = []
    for row in rows:
        items.append({"Tiempo_promedio": row[0]})

    conn.close()

    return jsonify(items)
```

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize JSON


```
1 [
2     {
3         "Tiempo_promedio": "00:19:45"
4     }
5 ]
```

UBER en SQL - Casos de uso (7)

```python
#¿Cuales son los usuarios que han dado una calificacion de 5 o una calificacion menor de 2?
@app.route('/users/fivestars', methods=['GET'])
def obtener_usuarios_calificaciones():
    conn = get_db_connection()
    cursor = conn.cursor()

    cursor.execute("""SELECT DISTINCT id_usuario
                      FROM Reseña
                      WHERE calificacion = 5 OR calificacion <= 2;
                      """)
    rows = cursor.fetchall()

    items = []
    for row in rows:
        items.append({"id_usuario": row[0]})

    conn.close()

    return jsonify(items)
```

UBER en SQL - Casos de uso (7)

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize JSON


```
[
    {
        "id_usuario": 1
    },
    {
        "id_usuario": 2
    },
    {
        "id_usuario": 4
    }
]
```

UBER en Mongo Caso de uso (1)

```
[
  {
    _id: ObjectId('672fd0c88158a9d1c10d81f4'),
    nombre: 'Juan',
    apellido: 'Pérez',
    email: 'juan.perez@example.com',
    telefono: '123456789',
    'reseñasDadas': [
      {
        id_viaje: ObjectId('672fca1c8158a9d1c10d81ef'),
        calificacion: 5,
        comentario: 'Excelente servicio!',
        fecha: ISODate('2024-10-01T00:00:00.000Z')
      }
    ],
    pagos: [
      {
        id_viaje: ObjectId('672fca1c8158a9d1c10d81ef'),
        fechaPago: ISODate('2024-10-01T00:00:00.000Z'),
        total: 1500,
        estado: 'Completado',
        metodoDePago: 'Tarjeta'
      }
    ]
  },
```

UBER en Mongo Caso de uso (1)

```python
#Endpoints mongo

#¿Cual es el top 3 de usuarios que han dado mas reseñas?
@app.route('/mongo/users/top3', methods=['GET'])
def mongo_obtener_top_usuarios():
    pipeline = [
        {"$project": {"nombre": 1, "apellido": 1, "email": 1, "cantidad_reseñas": {"$size": "$reseñasDadas"}}},
        {"$sort": {"cantidad_reseñas": -1}},
        {"$limit": 3}
    ]
    usuarios = list(db.usuarios.aggregate(pipeline))
    for usuario in usuarios:
        usuario["_id"] = objectid_to_str(usuario["_id"])

    return jsonify(usuarios)
```

UBER en Mongo Caso de uso (1)

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize JSON


```
[
    {
        "_id": "672fd0c88158a9d1c10d81f5",
        "apellido": "Gómez",
        "cantidad_reseñas": 2,
        "email": "maria.gomez@example.com",
        "nombre": "María"
    },
    {
        "_id": "672fd0c88158a9d1c10d81f4",
        "apellido": "Pérez",
        "cantidad_reseñas": 1,
        "email": "juan.perez@example.com",
        "nombre": "Juan"
    },
    {
        "_id": "672fd0c88158a9d1c10d81f6",
        "apellido": "López",
        "cantidad_reseñas": 1,
        "email": "carlos.lopez@example.com",
        "nombre": "Carlos"
    }
]
```

UBER en Mongo Caso de uso (2)

```python
#¿Cual es el metodo de pago menos utilizado en la plataforma?
@app.route('/mongo/pay-method/lessused', methods=['GET'])
def mongo_obtener_metodo_pago_menos_utilizado():
    pipeline = [
        {"$unwind": "$pagos"},
        {"$group": {"_id": "$pagos.metodoDePago", "cantidad": {"$sum": 1}}},
        {"$sort": {"cantidad": 1}},
        {"$limit": 1}
    ]
    resultado = list(db.usuarios.aggregate(pipeline))
    return jsonify(resultado)
```

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize JSON

```
1 [
2     {
3         "_id": "Tarjeta",
4         "cantidad": 2
5     }
6 ]
```

UBER en Mongo Caso de uso (3)

```
[
  {
    _id: ObjectId('672fc5988158a9d1c10d81d5'),
    nombre: 'Carlos',
    apellido: 'Sánchez',
    licencia: 'ABC123456',
    telefono: '5551234567',
    email: 'carlos.sanchez@example.com',
    ultima_actividad: ISODate('2024-11-01T11:30:00.000Z')
  },
  {
```

UBER en Mongo Caso de uso (3)

```python
#¿Cuales son los conductores que han estado inactivos en el ultimo mes?
@app.route('/mongo/drivers/inactive', methods=['GET'])
def mongo_obtener_conductores_inactivos():
    fecha_limite = datetime.now() - timedelta(days=30)
    pipeline = [
        {"$match": {"ultima_actividad": {"$lt": fecha_limite}}},
        {"$project": {"nombre": 1, "apellido": 1, "ultima_actividad": 1}}
    ]
    conductores_inactivos = list(db.conductores.aggregate(pipeline))

    for conductor in conductores_inactivos:
        conductor["_id"]= objectid_to_str(conductor["_id"])

    return jsonify(conductores_inactivos)
```

UBER en Mongo Caso de uso (3)

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize JSON


```
[
    {
        "_id": "672fc6f38158a9d1c10d81d8",
        "apellido": "Martinez",
        "nombre": "Roberto",
        "ultima_actividad": "Thu, 03 Oct 2024 17:00:00 GMT"
    },
    {
        "_id": "672fc6f38158a9d1c10d81d9",
        "apellido": "Ramirez",
        "nombre": "Claudia",
        "ultima_actividad": "Fri, 04 Oct 2024 12:00:00 GMT"
    },
    {
        "_id": "672fc6f38158a9d1c10d81da",
        "apellido": "Sánchez",
        "nombre": "Esteban",
        "ultima_actividad": "Sat, 05 Oct 2024 21:00:00 GMT"
    }
]
```

UBER en Mongo Caso de uso (4)

```
{
  _id: ObjectId('672fca1c8158a9d1c10d81ef'),
  id_conductor: ObjectId('672fc5988158a9d1c10d81d5'),
  id_vehiculo: ObjectId('672fc7208158a9d1c10d81e0'),
  punto_partida: 'Av. Libertador 5000, Buenos Aires',
  destino: 'Calle 25 de Mayo 1200, Buenos Aires',
  distancia: 10,
  tiempo_viaje: 15,
  estado: 'Completado',
  fecha_hora_inicio: ISODate('2024-10-01T11:00:00.000Z'),
  fecha_hora_final: ISODate('2024-10-01T11:15:00.000Z')
},
{
  _id: ObjectId('672fca1c8158a9d1c10d81f0'),
  id_conductor: ObjectId('672fc5988158a9d1c10d81d6'),
  id_vehiculo: ObjectId('672fc7208158a9d1c10d81e1'),
  punto_partida: 'Av. Corrientes 3000, Buenos Aires',
  destino: 'Calle San Martín 4000, Buenos Aires',
  distancia: 8,
  tiempo_viaje: 12,
  estado: 'Completado',
  fecha_hora_inicio: ISODate('2024-10-01T12:00:00.000Z'),
  fecha_hora_final: ISODate('2024-10-01T12:12:00.000Z')
},
```

UBER en Mongo Caso de uso (4)

```python
#¿Cual es el tiempo promedio de los viajes?
@app.route('/mongo/travels/avgtime', methods=['GET'])
def mongo_obtener_tiempo_promedio():
    pipeline = [
        {"$group": {"_id": None, "tiempo_promedio": {"$avg": "$tiempo_viaje"}}}
    ]
    resultado = list(db.viajes.aggregate(pipeline))
    return jsonify(resultado)
```

Pretty | Raw | Preview | Visualize | JSON


```json
[
    {
        "_id": null,
        "tiempo_promedio": 10.4
    }
]
```

UBER en Mongo Caso de uso (5)

```
#¿Que conductores y pasajeros han coincidido en mas de 1 viaje en la aplicacion?
@app.route('/mongo/coincidencias', methods=['GET'])
def mongo_obtener_viajes_comunes():
    pipeline = [
        {"$lookup": {
            "from": "usuarios",
            "localField": "_id",
            "foreignField": "reseñasDadas.id_viaje",
            "as": "pasajeros"
        }},

        {"$unwind": "$pasajeros"},
        {"$group": {
            "_id": {
                "conductor": "$id_conductor",
                "pasajero": "$pasajeros._id"
            },
            "viajes_comunes": {"$sum": 1}
        }},

        {"$match": {"viajes_comunes": {"$gt": 1}}},

        {"$lookup": {
            "from": "conductores",
            "localField": "_id.conductor",
            "foreignField": "_id",
            "as": "conductor_info"
        }},
```

UBER en Mongo Caso de uso (5)

```
    {"$lookup": {
        "from": "usuarios",
        "localField": "_id.pasajero",
        "foreignField": "_id",
        "as": "pasajero_info"
    }},

    {"$project": {
        "conductor": {"$arrayElemAt": ["$conductor_info.nombre", 0]},
        "pasajero": {"$arrayElemAt": ["$pasajero_info.nombre", 0]},
        "viajes_comunes": 1
    }}
]


coincidencias = list(db.viajes.aggregate(pipeline))

for coincidencia in coincidencias:
    if "_id" in coincidencia and "conductor" in coincidencia["_id"]:
        coincidencia["_id"]["conductor"] = objectid_to_str(coincidencia["_id"]["conductor"])
    if "_id" in coincidencia and "pasajero" in coincidencia["_id"]:
        coincidencia["_id"]["pasajero"] = objectid_to_str(coincidencia["_id"]["pasajero"])

return jsonify(coincidencias)
```

UBER en Mongo Caso de uso (5)

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize JSON


```
[
    {
        "id_coductor": 1,
        "id_usuario": 1,
        "total_viajes": 3
    }
]
```

UBER en Mongo Caso de uso (6)

```python
#¿Cual es el tiempo promedio de los viajes?
@app.route('/mongo/travels/avgtime', methods=['GET'])
def mongo_obtener_tiempo_promedio():
    pipeline = [
        {"$group": {"_id": None, "tiempo_promedio": {"$avg": "$tiempo_viaje"}}}
    ]
    resultado = list(db.viajes.aggregate(pipeline))
    return jsonify(resultado)
```

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize JSON


```
1 [
2     {
3         "_id": null,
4         "tiempo_promedio": 10.4
5     }
6 ]
```

UBER en Mongo Caso de uso (7)

```
#¿Cuales son los usuarios que han dado una calificacion de 5 o una calificacion menor de 2?
@app.route('/mongo/users/stars', methods=['GET'])
def mongo_obtener_usuarios_calificaciones():
    pipeline = [
        {"$unwind": "$reseñasDadas"},
        {"$match": {"$or": [{"reseñasDadas.calificacion": 5}, {"reseñasDadas.calificacion": {"$lte": 2}}]}},
        {"$project": {"nombre": 1, "apellido": 1, "email": 1, "reseñasDadas.calificacion": 1}}
    ]
    usuarios = list(db.usuarios.aggregate(pipeline))

    for usuario in usuarios:
        usuario["_id"] = objectid_to_str(usuario["_id"])

    return jsonify(usuarios)
```

UBER en Mongo Caso de uso(7)

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize JSON


```
[
    {
        "_id": "672fd0c68158a9d1c10d81f4",
        "apellido": "Pérez",
        "email": "juan.perez@example.com",
        "nombre": "Juan",
        "reseñasDadas": {
            "calificacion": 5
        }
    },
    {
        "_id": "672fd0c68158a9d1c10d81f7",
        "apellido": "Torres",
        "email": "ana.torres@example.com",
        "nombre": "Ana",
        "reseñasDadas": {
            "calificacion": 2
        }
    }
]
```

UBER en Cassandra Caso de uso

```
cqlsh:cassandratp> select * from Vehiculos ;

 marca      | ultima_letra_patente | patente | id_vehiculo | id_conductor | modelo
------------+----------------------+---------+-------------+--------------+--------
    Toyota  |                    3 |  ABC123 |           1 |            1 | Corolla
      Ford  |                    9 |  XYZ789 |           2 |            2 |  Fiesta
 Chevrolet  |                    6 |  LMN456 |           3 |            3 |    Onix
Volkswagen  |                    1 |  DEF321 |           4 |            4 |    Polo
     Honda  |                    4 |  GHI654 |           5 |            5 |   Civic
    Toyota  |                    D |  GHI65D |           5 |            5 |   Civic
```

UBER en Cassandra Caso de uso

```
#Cassandra: ¿Cuantos autos con terminacion de patente en "D" y de Marca Toyota hay en la plataforma?
@app.route('/cassandra/cars/toyota', methods=['GET'])
def cassandra_obtener_autos_toyota():
    session = get_cassandra_connection()

    query = SimpleStatement("""SELECT id_vehiculo, modelo, patente FROM Vehiculos
                               WHERE marca = 'Toyota' AND ultima_letra_patente = 'D';""")
    rows = session.execute(query)

    reseñas = []
    for row in rows:
        reseñas.append({
            'id_vehiculo': row.id_vehiculo,
            'modelo': row.modelo,
            'patente': row.patente
        })

    return jsonify(reseñas)
```

UBER en Cassandra Caso de uso

Body Cookies Headers (5) Test Results 200 OK • 179

Pretty Raw Preview Visualize JSON


```
[
    {
        "id_vehiculo": 5,
        "modelo": "Civic",
        "patente": "GHI65D"
    }
]
```


NOTA: somos concientes de que el Civic es Honda, pero en los inserts cometimos un pequeño error y quedo con marca Toyota.

UBER en Cassandra Caso de uso 2

```
cqlsh:tpuber> SELECT * FROM resenas;

 calificacion | id_resena | id_usuario
--------------+-----------+------------
            5 |         1 |          1
            2 |         2 |          1
            9 |         2 |          1

(3 rows)
cqlsh:tpuber> SELECT * FROM resenas WHERE calificacion IN (1,5);

 calificacion | id_resena | id_usuario
--------------+-----------+------------
            5 |         1 |          1

(1 rows)
cqlsh:tpuber>
```

Conclusiones

Comparaciones y conclusiones sobre las diferencias y desafíos encontrados al implementar el mismo diseño en diferentes tecnologías.

SQL garantiza consistencia y precisión a través de transacciones ACID,los datos deben ser precisos y seguros, además facilita el uso de relaciones con los JOINS. Pero esta limitada para grandes volúmenes de datos.

MongoDB su modelo de documentación es flexible permitiendo cambios en el esquema sin mucha dificultad, también es ideal para trabajar con grandes volúmenes de datos. Pero las relaciones complejas no se gestionan bien y puede provocar datos duplicados

Cassandra me permite gestionar grandes volúmenes de datos distribuidos geográficamente con una consistencia ajustable según la necesidad de cada, pero no es adecuada para consultas que requieran datos precisos y actualizados y no soporta JOINS

Final

Consultas?